

FinGPT Part 2 开发文档

这份文档是 `DEVELOPMENT.md` 的中文版本，重点说明当前仓库的真实 workflow、每一次 LLM 调用分别在做什么，以及回测系统如何把两阶段模型串起来。

项目目标

这个项目实现了一条本地运行的金融新闻信号流水线：

1. **Agent 1** 把原始新闻文本变成结构化指纹 `NewsFingerprint`
2. **Agent 2** 基于指纹生成交易方向 `TradingSignal`
3. **backtest** 模块把同样的流程跑在历史数据集上，并与真实收益对比

这个仓库最核心的设计不是让模型“自己报分数”，而是通过 vLLM 的 `prompt_logprobs` 直接读取模型在决策点上的真实 token 对数概率，再由 Python 做确定性的后处理。

整体 workflow

当前主要有两种运行模式。

1. 实时抓取模式

入口文件是 `pipeline.py`。

流程如下：

1. 调用 `run_pipeline(tickers, limit, as_of_timestamp)`
2. 调用 `fetch_recent_articles(...)` 从 Alpaca 或 Finnhub 抓候选新闻
3. 在抓取层做 leakage-safe 过滤：
 - 只保留 `created_at <= as_of_timestamp` 的新闻
 - 丢掉没有时间戳或时间戳无法解析的新闻
 - 对每个 ticker 只保留该时点之前最近的一篇
4. 对保留下来的每篇新闻：
 - 构造 `article_text = headline + summary`
 - 调用 `extract_fingerprint(article_text, ticker=..., headline=...)`
 - 再调用 `generate_signal(fingerprint)`
5. 把有效信号写入 `output/signals_<timestamp>.json`

实时模式下有两个重要规则：

- `ticker` 由抓取层提供，是外部确定信息
- `headline` 也由抓取层提供，是外部确定信息
- **Agent 2** 不会重新阅读新闻正文

2. 回测模式

入口文件是 `backtest/backtester.py`。

流程如下：

1. 读取数据集，可以是：
 - Hugging Face dataset
 - 本地 `.csv`
 - 本地 `.parquet`
2. 统一列名为：
 - `input`
 - `output`
 - `answer`
 - `ticker`
3. 解析成回测行，每行包含：
 - `ticker`
 - `headline`
 - `article_text`
 - `start_date`
 - `end_date`
 - `fingpt_label`
4. 以 batch 的方式运行：
 - Agent 1 批量提取
 - Agent 2 批量打信号
 - 真实收益抓取
 - 行级结果拼装
5. 输出详细 CSV

回测模式下有一个特别重要的规则：

- 对 HF 数据集中一条 prompt 里出现多篇新闻的情况，当前会取第一篇 headline 作为该样本的原始 `headline`
- 数据集里的 `ticker` 会直接传给 Agent 1，作为权威 ticker

核心数据结构

NewsFingerprint

定义在 [agent1/schema.py](#)。

字段分为几组：

元数据 / 上游传入字段

- `ticker`
- `source`
- `published_at`
- `headline`
- `companies_named`
- `event_keywords`

情绪字段

- `sentiment_label`

- `sentiment_score`
- `sentiment_confidence`
- `sentiment_probabilities`
- `sentiment_logits`
- `calibration_T`

事件类型字段

- `event_type`
- `event_type_confidence`
- `event_type_margin`
- `event_type_method`
- `event_type_logits`
- `event_type_probabilities`
- `secondary_event_type`
- `secondary_event_type_confidence`

透传字段

- `article_text`

几个关键行为：

- `ticker` 现在不再依赖 `companies_named[0]`
- `headline` 优先保留上游传入的原始 `headline`
- `source` / `published_at` 如果模型抽成了 `list`，会在组装前归一化成单个字符串
- `companies_named` 如果为空，会尽量用上游 `ticker` 补进去
- `event_keywords` 现在主要是兼容字段，组装时会被设置成 `[event_type.lower()]`

TradingSignal

定义在 [agent2/schema.py](#)。

核心字段包括：

- `ticker`
- `direction`
- `strategy_tag`
- `confidence`
- `cot`
- `signal_logits`
- `raw_signal_logits`
- `pmi_null_logprobs`
- `pmi_alpha_used`
- `signal_probabilities`
- `calibration_T`
- `signal_filter_forced_hold`
- `signal_filter_reason`

其中：

- `strategy_tag` 目前固定为 `"event_driven"`
- `cot` 在 no-CoT 模式下是空字符串

每一次 LLM 调用到底在做什么

这一节是最重要的。

Agent 1 调用 1: guided fact extraction

代码位置：

- `agent1/extractor.py`

Prompt 来源：

- `agent1/prompt.py` 中的 `EXTRACTION_PROMPT`

输入：

- 原始 `article_text`

要求模型输出的结构化字段：

- `source`
- `published_at`
- `companies_named`
- `event_keywords`

执行机制：

1. 用 `EXTRACTION_PROMPT + article_text` 组成 extraction prompt
2. 用 guided schema 要求模型输出 JSON
3. 解析顺序是：
 - 先按 JSON 解析
 - 再尝试抽平衡的大括号 JSON
 - 再尝试 markdown 风格 fallback

当前的稳健性处理：

- 如果 extraction 输出完全坏掉，已经不会直接让整条样本失败
- 现在会退化成一个空的 extraction payload：
 - `source=""`
 - `published_at=""`
 - `headline=""`
 - `companies_named=[]`
 - `event_keywords=[]`

为什么这样仍然可以继续回测：

- `source` 不影响 Agent 2

- `headline` 通常已有外部传入值
- `ticker` 通常已有外部传入值
- `companies_named` 可以用 `ticker` fallback

Agent 1 调用 2: sentiment scoring

代码位置:

- `agent1/extractor.py` 中的 `_score_sentiment`

Prompt 来源:

- `agent1/prompt.py` 中的 `SENTIMENT_PROMPT`

输入:

- 原始 `article_text`

模型候选输出标签:

- `POSITIVE`
- `NEGATIVE`
- `NEUTRAL`

执行机制:

1. 构造直接分类的 sentiment prompt
2. 调 `get_real_choice_logits(...)` 或 batch 版本
3. 指定:
 - `decision_prefix = "Sentiment: "`
 - `use_cot = False`
4. 从决策点读取三个类标签的真实 token log-probabilities
5. 用 `softmax(logits / CALIBRATION_T)` 计算概率
6. 生成:
 - `sentiment_label`
 - `sentiment_confidence`
 - `sentiment_probabilities`
 - `sentiment_logits`

当前的降级逻辑:

- 如果 sentiment logits 解析失败
- 不会让整条样本中断
- 会退化成:
 - 均匀概率
 - `NEUTRAL`
 - 零 logits

Agent 1 调用 3: event_type scoring

代码位置:

- `agent1/extractor.py` 中的 `_score_event_type`

Prompt 来源:

- `agent1/prompt.py` 中的 `EVENT_TYPE_PROMPT`

输入:

- 原始 `article_text`

模型评分 token:

- `A, B, C, D, E, F, G`

映射关系:

- `A -> EARNINGS`
- `B -> GUIDANCE`
- `C -> ANALYST_RATING`
- `D -> LEGAL_REGULATORY`
- `E -> MNA`
- `F -> PRODUCT_BUSINESS`
- `G -> MACRO`

执行机制:

1. 构造 event-type prompt
2. 调 `get_real_choice_logits(...)` 或 batch 版本
3. 指定:
 - `decision_prefix = "Answer: "`
 - `use_cot = False`
4. 读取 A-G 的真实 token log-probabilities
5. 用 `softmax(logits / CALIBRATION_T)` 做概率校准
6. 找 top-1 / top-2
7. 应用规则过滤:
 - 如果 `top_prob < FINGPT_EVENT_TYPE_MIN_CONFIDENCE`, 输出 `OTHER`
 - 如果 `margin < FINGPT_EVENT_TYPE_MIN_MARGIN`, 输出 `OTHER`
 - 否则接受 top 类别

重要设计:

- `OTHER` 不是模型打分出来的
- `OTHER` 只由 Python 规则引擎赋值

Agent 2 调用 1: PMI null-context prior

代码位置:

- `agent2/reasoner.py` 中的 `_compute_null_logprobs`

作用:

- 计算在没有真实新闻语境时，模型对 A/B/C 三个策略 token 的先验偏好

执行机制：

1. 构造一个合成的中性 `NewsFingerprint`
2. 构造 Agent 2 prompt
3. 对 A/B/C 打分
4. 结果存入：
 - 进程内缓存
 - 可选的磁盘缓存

这个调用不是每篇新闻都跑，而是每组模型/配置组合只跑一次。

Agent 2 调用 2：strategy scoring

代码位置：

- `agent2/reasoner.py`

Prompt 来源：

- `agent2/prompt.py`

Agent 2 实际看到的输入：

- `ticker`
- `headline`
- Agent 1 输出的 `sentiment` 字段
- Agent 1 输出的 `event_type` 字段
- `companies_named`

Agent 2 明确看不到：

- 完整 `article_text`
- 新闻正文
- 旧的 event keyword 抽取结果

模型评分 token：

- `A, B, C`

映射关系：

- `A -> BUY -> long`
- `B -> HOLD -> neutral`
- `C -> SELL -> short`

执行机制：

1. 构造 compact fingerprint-only prompt
2. 如果 `FINGPT_SIGNAL_USE_COT=True`
 - 先生成 CoT
 - 再对 A/B/C 打分

3. 否则直接对 A/B/C 打分
4. 如果启用 PMI:
 - `adjusted = raw - pmi_alpha * null`
5. 用 `softmax(adjusted / CALIBRATION_T)` 计算最终概率
6. 应用规则过滤:
 - 低 confidence -> 强制 HOLD
 - 低 margin -> 强制 HOLD
 - BUY 不够强 -> 强制 HOLD
 - SELL 不够强 -> 强制 HOLD
7. 组装 `TradingSignal`

Batch 行为

Agent 1 batch

`extract_fingerprint_batch(article_texts, tickers=None, headlines=None)` 会执行:

1. 一次 batched guided extraction 调用
2. 一次 batched sentiment scoring 调用
3. 一次 batched event-type scoring 调用
4. Python 侧逐条组装 fingerprint

现在即使某一条 extraction JSON 坏掉, 逐条组装时也尽量不要立刻失败, 因为 extraction 已经有 empty-payload fallback。

Agent 2 batch

`generate_signal_batch(fingerprints)` 会执行:

- no-CoT 模式:
 - 一次 batched A/B/C scoring
- CoT 模式:
 - 一次 batched CoT generation
 - 一次 batched A/B/C scoring

Backtest batch

一个 batch 的实际顺序是:

1. 跑 Agent 1 batch extraction
2. 过滤出有效 fingerprints
3. 跑 Agent 2 batch signal generation
4. 逐行抓 realized return
5. 拼装成最终 CSV

失败与降级逻辑

Fact extraction 失败

当前行为:

- extraction JSON 坏掉时，不会自动整条中断
- 会退化为空 extraction payload
- 只要 sentiment / event_type 正常，这条样本仍然可以继续进入 Agent 2

Sentiment 失败

当前行为：

- 降级成 **NEUTRAL**
- 概率均匀

Event-type 失败

当前行为：

- 赋值为 **OTHER**
- `event_type_method = "event_type_logits_failed"`

Agent 2 失败

当前行为：

- 返回 **None**
- 回测里标记成 **signal_failed**

配置项

主配置在 [config.py](#)。

核心模型 / 推理

- **FINGPT_MODEL_PATH**
- **SHARE_SINGLE_LLM_BETWEEN_AGENTS**
- **CALIBRATION_T**
- **LOGITS_MAX_TOKENS**

新闻抓取

- **NEWS_PROVIDER**
- **ALPACA_API_KEY**
- **ALPACA_API_SECRET**
- **ALPACA_DEFAULT_LIMIT**
- **FINGPT_NEWS_FETCH_COUNT**
- **FINNHUB_API_KEY**
- **FINNHUB_TIMEOUT_SEC**
- **FINNHUB_MAX_CALLS_PER_SEC**
- **FINNHUB_MAX_RETRIES**
- **FINNHUB_RETRY_BASE_DELAY_SEC**

Agent 1 event_type 过滤

- FINGPT_EVENT_TYPE_MIN_CONFIDENCE
- FINGPT_EVENT_TYPE_MIN_MARGIN

Agent 2 PMI / 信号过滤

- FINGPT_PMI_ALPHA
- FINGPT_SIGNAL_MIN_CONFIDENCE
- FINGPT_SIGNAL_MIN_MARGIN
- FINGPT_BUY_THRESHOLD
- FINGPT_SELL_THRESHOLD
- FINGPT_SIGNAL_USE_COT

回测

- FINGPT_BACKTEST_STRICT_MODE
- FINGPT_BACKTEST_BATCH_SIZE

回测输出

主要输出是一个详细 CSV，每一行对应一条数据集样本。

关键暴露字段包括：

原始 / 元数据

- ticker
- headline
- fingerprint_ticker
- signal_ticker
- start_date
- end_date
- fingpt_label

Agent 1

- sentiment_*
- event_type_*
- event_logprob_*
- event_prob_*

Agent 2

- direction
- confidence
- raw_signal_logprob_*
- pmi_null_logprob_*
- pmi_adjusted_logit_*
- signal_prob_*
- signal_filter_*

评估

- [realized_return](#)
- [strategy_return](#)
- [skipped_reason](#)

想改哪里就看哪里

想改抽取逻辑

- [agent1/prompt.py](#)
- [agent1/extractor.py](#)
- [agent1/schema.py](#)

想改策略信号逻辑

- [agent2/prompt.py](#)
- [agent2/reasoner.py](#)
- [agent2/schema.py](#)

想改回测

- [backtest/dataset_parser.py](#)
- [backtest/backtester.py](#)
- [backtest/price_fetcher.py](#)

想改实时抓取 / leakage-safe 逻辑

- [ingestion/news_fetcher.py](#)
- [pipeline.py](#)